
System call and User Program

Adding a system call and a usertest

Contents

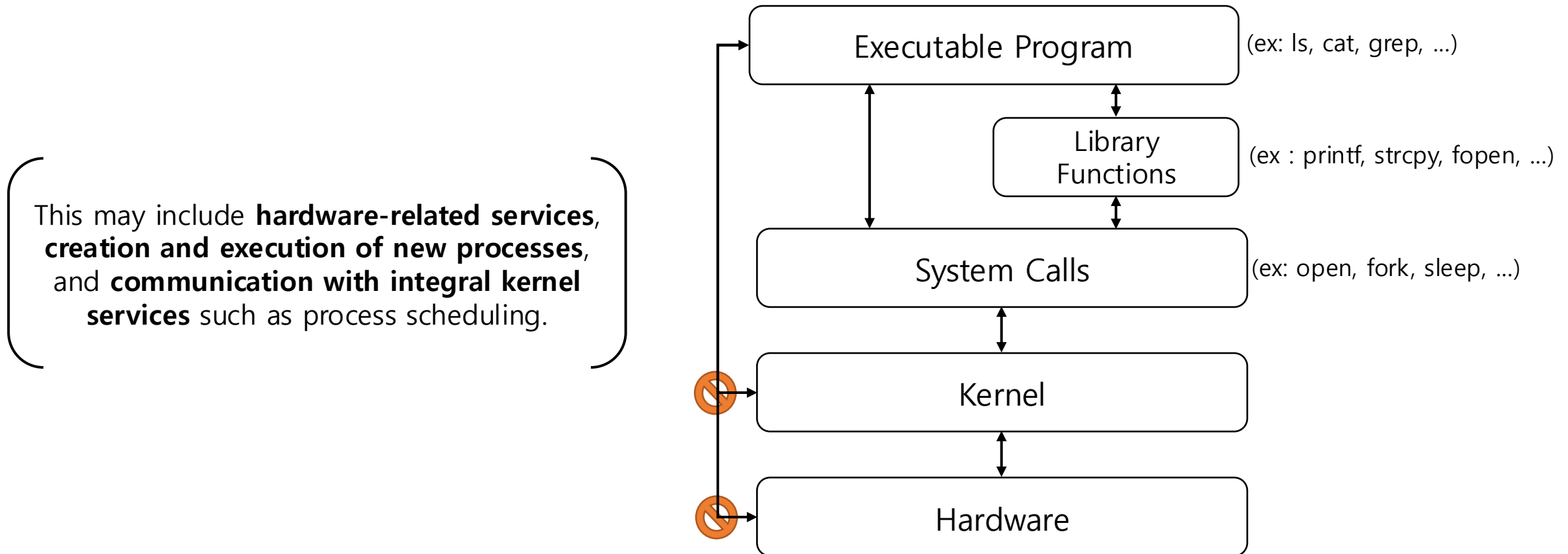
- What is system call?
- xv6 system call flow
- **Hands-on Labs**
 - Add a new system call: getppid
 - Add a new test program: getppidtest

System Call Overview

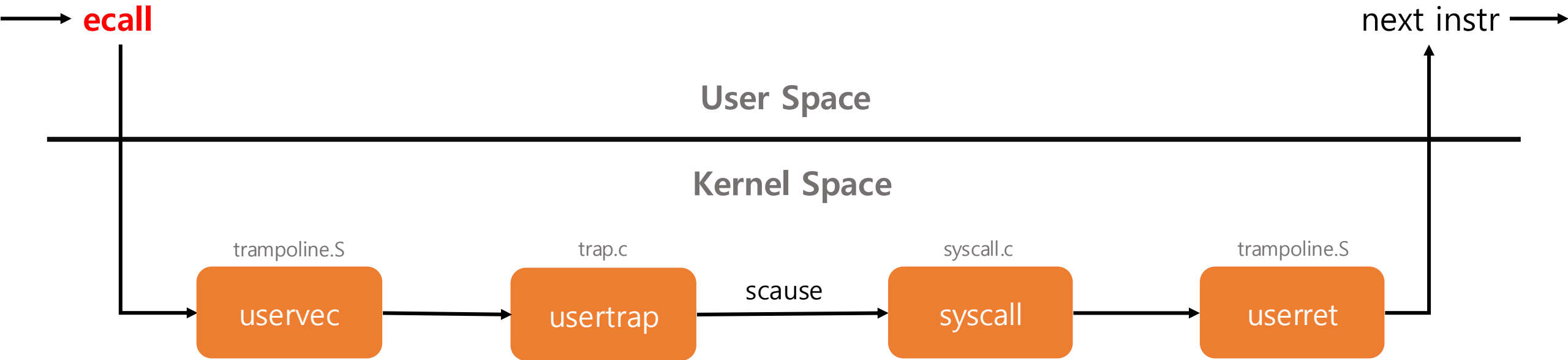
How does it work?

What is a System Call?

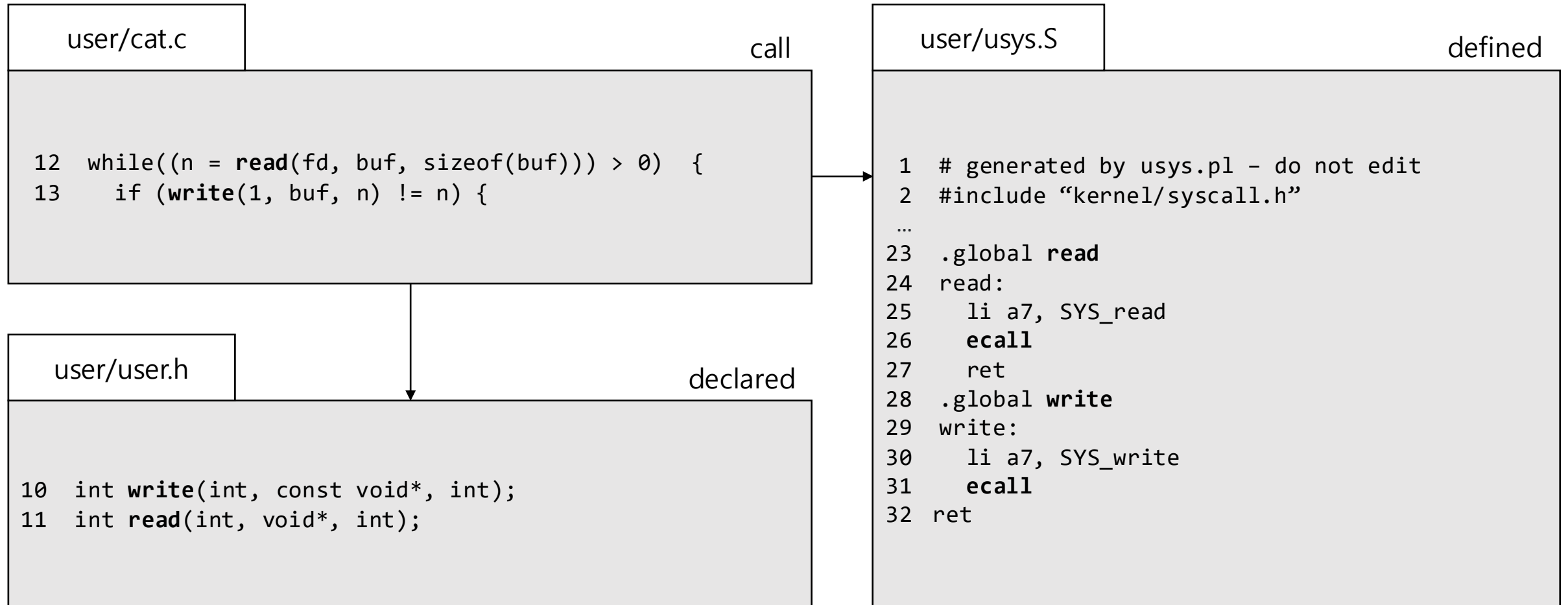
- A system call is the programmatic way in which a computer program requests a service from the kernel of the operating system it is executed on.



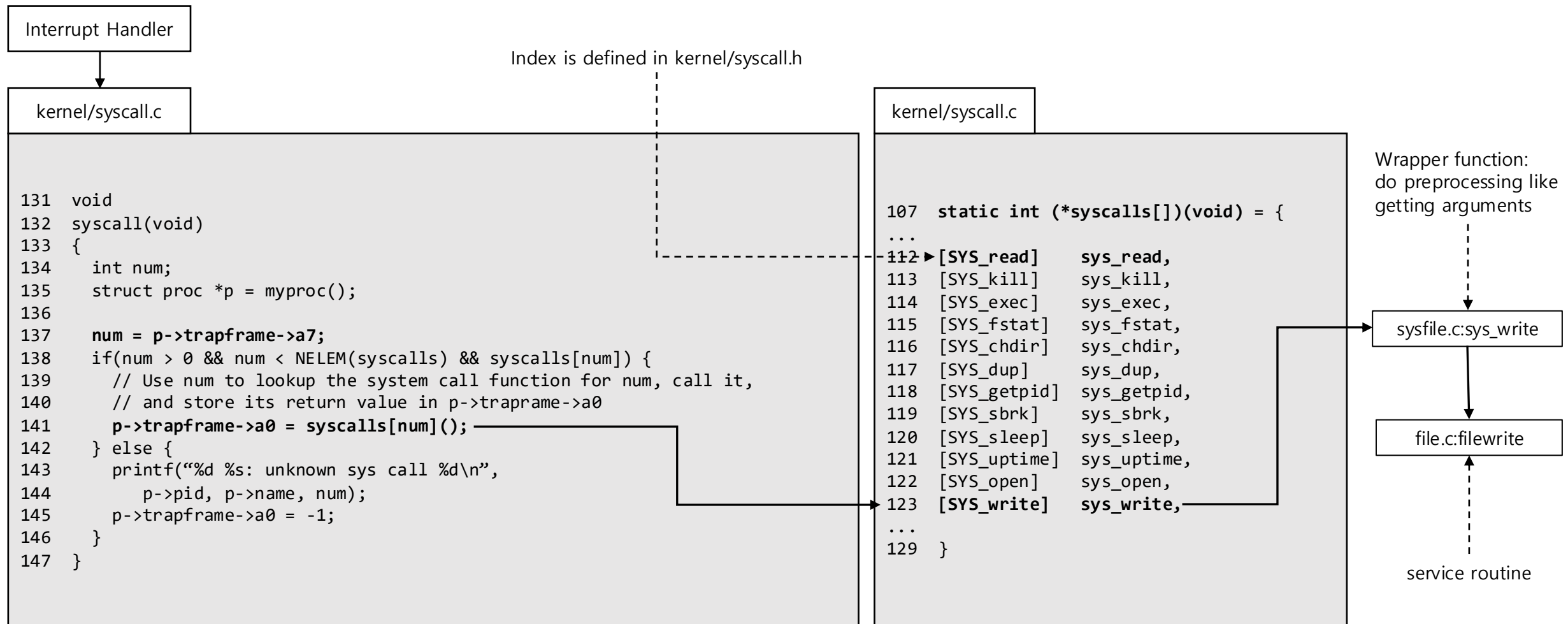
System Call Flow



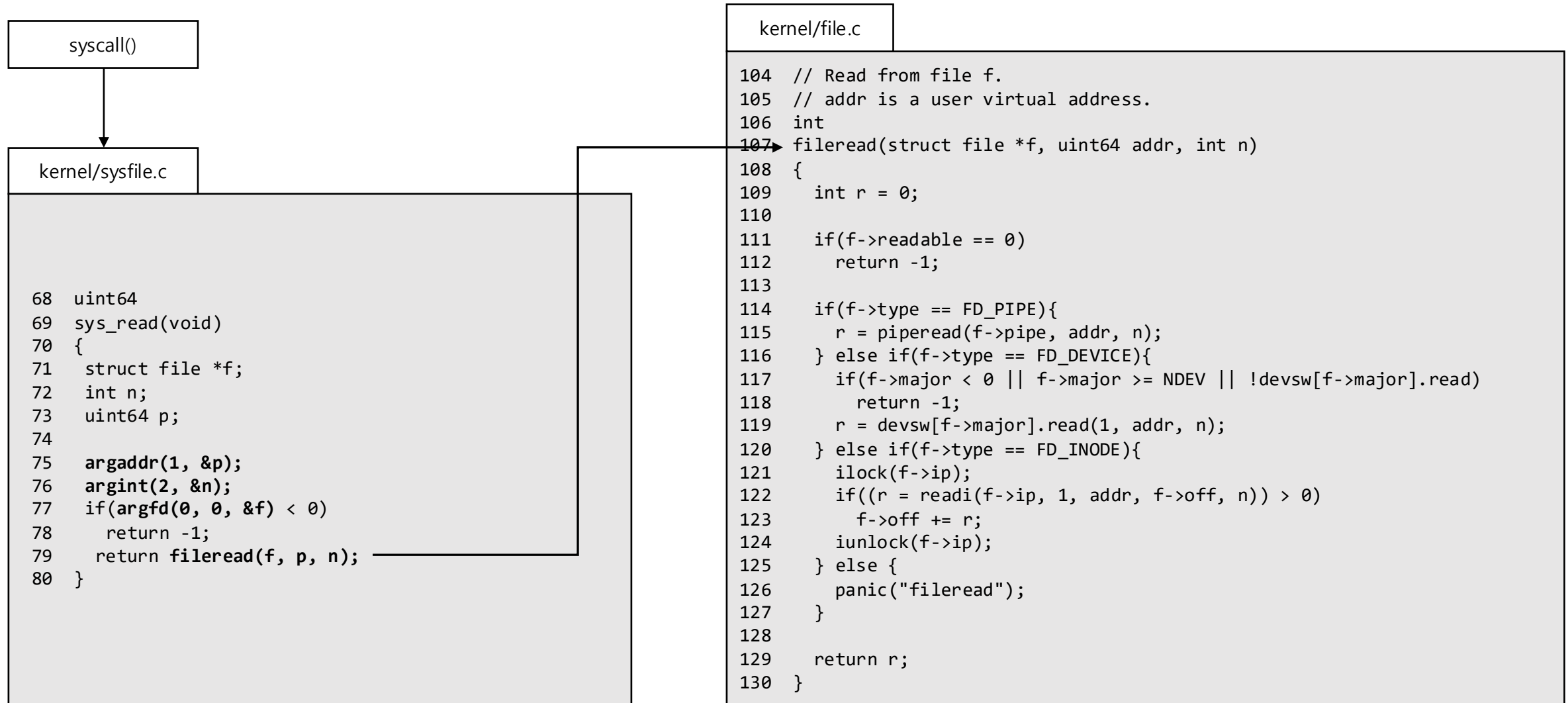
User: How to call a system call



Kernel: How to handle system call



Kernel: (wrapper) function

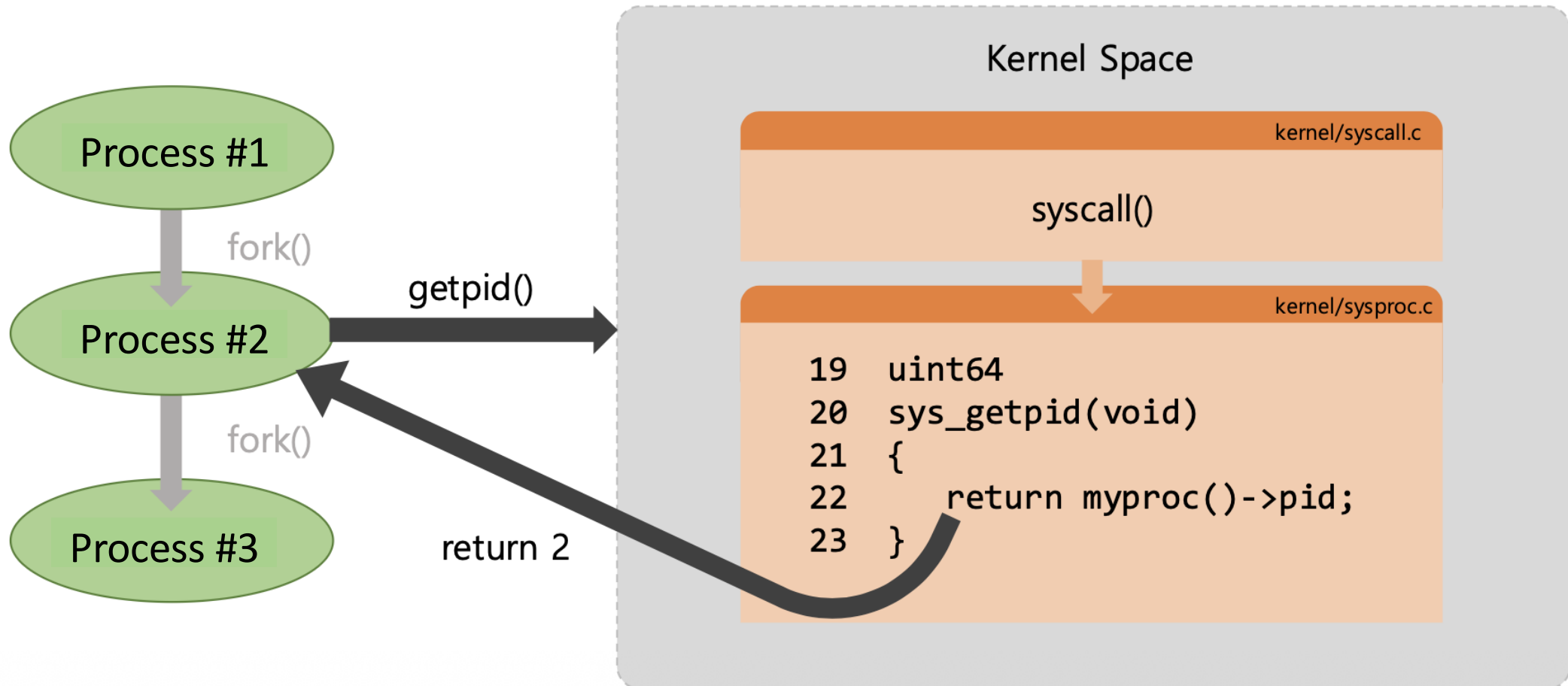


Add a New System Call

Who is your parent?

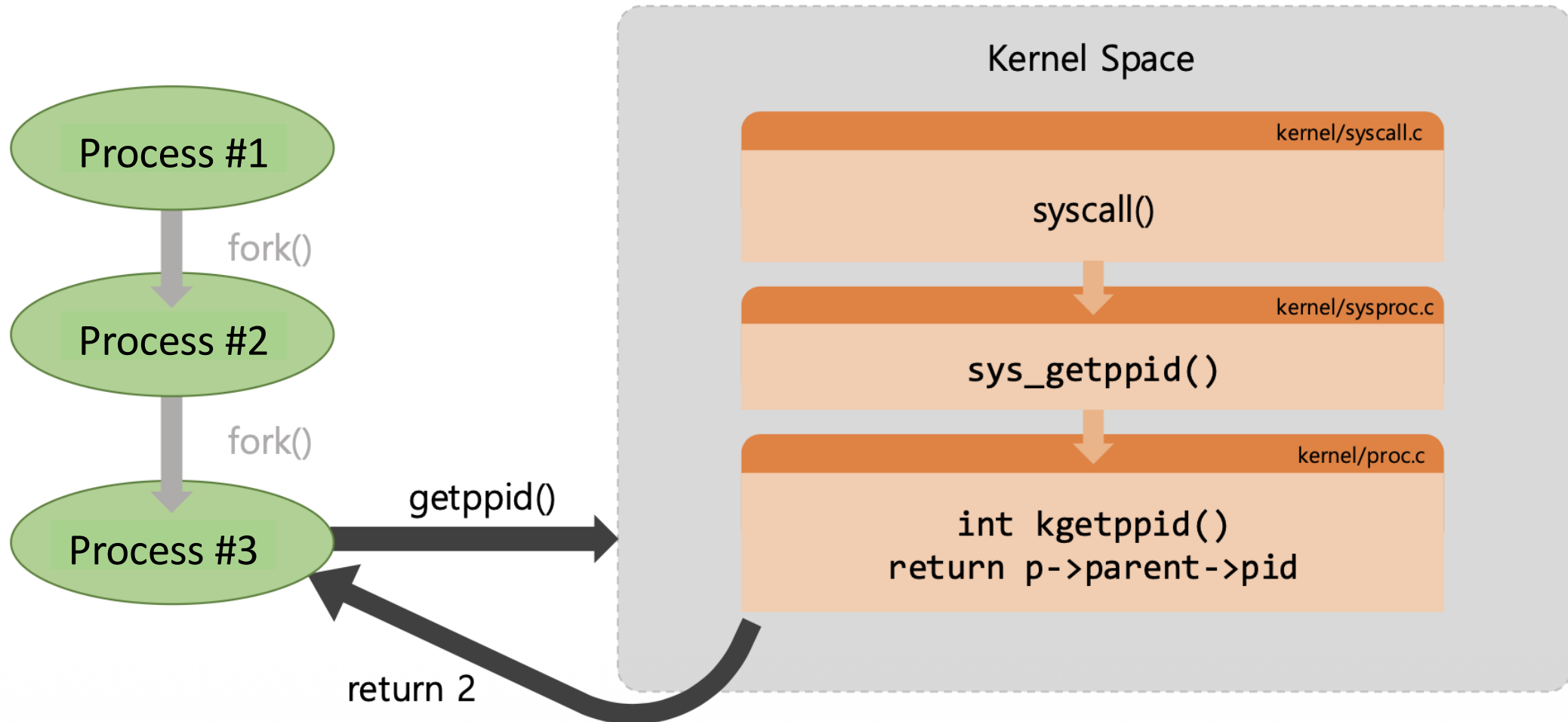
Existing System Call: getpid()

- getpid returns the **current** process ID



Our Goal: Adding getppid()

- Let's add a new system call that returns the **parent** process ID



How to add a new system call

- Kernel

1. Implement a function
2. Declare the function in `kernel/defs.h`
3. Implement a wrapper function to call the function
4. Register the new system call in `kernel/syscall.h` and `kernel/syscall.c`

- User

1. Declare the function in `user/user.h`
2. Add a new macro in `user/usys.pl`

Step 0 – git branch

- Before we start, let's make a branch named 'sandbox/lab03'

```
$ git switch sandbox/base      # switch to base branch
$ git branch sandbox/lab03    # make a new branch
$ git switch sandbox/lab03    # switch to lab03 branch
```

- TA's code will be uploaded this weekends in [sandbox/lab03 branch on TA's repository](#)

Step 1 – Implement a function

- **kernel**/proc.c contains process-related code
- Implement our new kgetppid function in **kernel**/proc.c

```
$ vim kernel/proc.c

692  int
693  kgetppid(void)
694  {
695      struct proc *p = myproc();
696      if (p->parent) {
697          return p->parent->pid;
698      } else {
699          return -1;
700      }
701  }
```

Step 2 – Declare the function in defs.h

- Add the following codes in **kernel**/defs.h
- In this step, it makes your function visible to other c files

```
$ vim kernel/defs.h

1  struct buf;
2  struct context;
3  struct file;
...
101 int    either_copyout(int user_dst, uint dst, ...
102 int    either_copyin(void *dst, int user_src, ...
103 void    procdump(void);
104 int    kgetppid(void);
```

Step 3 – Implement a wrapper function

- **kernel**/sysproc.c contains wrapper functions for process-related system calls
 - The wrapper function retrieves parameters from the trapframe.
 - However, there's no parameter to retrieve in this system call.
- Append the following code in **kernel**/sysproc.c

```
$ vim kernel/sysproc.c

111  uint64
112  sys_getppid(void)
113  {
114      return kgetppid();
115  }
```


Step 4 – Register the new system call

- Insert the following code in **kernel**/syscall.h and **kernel**/syscall.c
- In this step, you register your wrapper function as system call

```
$ vim kernel/syscall.h
```

```
1 // System call numbers
2 #define SYS_fork      1
3 #define SYS_exit      2
...
22 #define SYS_close     21
23 #define SYS_getppid    22
```

```
$ vim kernel/syscall.c
```

```
1 #include "types.h"
2 #include "param.h"
...
103 extern uint64 sys_close(void);
104 extern uint64 sys_getppid(void);
...
129 [SYS_close]      sys_close,
130 [SYS_getppid]     sys_getppid,
131 };
```

Step 5 – Declare the function in user.h

- Add the following codes in **user**/user.h
- In this step, it makes your system call visible to user program

```
$ vim user/user.h

1  struct stat;
2
3  // system calls
4  int fork(void);
5  int exit(int) __attribute__((noreturn))
...
26 int uptime(void);
27 int getppid(void);
```

Step 6 – Add new macro in usys.pl

- Append the following code in **user**/usys.pl
- In this step, it defines how to generate **user**/usys.S which call kernel's system call handling functions.

```
$ vim user/usys.pl
```

```
1  #!/usr/bin/perl -w
2
3  # Generate usys. S, the stubs for syscalls.
4
5  print "# generated by usys.pl - do not edit\n";
...
43 entry("pause");
44 entry("uptime");
45 entry("getppid");
```

```
104  uptime:
105      li a7, SYS_uptime
106      ecall
107      ret
108  .global getppid
109  getppid:
110      li a7, SYS_getppid
111      ecall
112      ret
```

```
// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
[SYS_fork]      sys_fork,
[SYS_exit]      sys_exit,
[SYS_wait]      sys_wait,
[SYS_pipe]      sys_pipe,
[SYS_read]      sys_read,
[SYS_kill]      sys_kill,
[SYS_exec]      sys_exec,
[SYS_fstat]     sys_fstat,
[SYS_chdir]     sys_chdir,
[SYS_dup]       sys_dup,
[SYS_getpid]    sys_getpid,
[SYS_sbrk]      sys_sbrk,
[SYS_pause]     sys_pause,
[SYS_uptime]    sys_uptime,
[SYS_open]      sys_open,
[SYS_write]     sys_write,
[SYS_mknod]     sys_mknod,
[SYS_unlink]    sys_unlink,
[SYS_link]      sys_link,
[SYS_mkdir]     sys_mkdir,
[SYS_close]     sys_close,
[SYS_getppid]   sys_getppid,
};
```

We are done! But...

```
$ make qemu
```

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
```

```
xv6 kernel is booting
```

```
hart 2 starting
```

```
hart 1 starting
```

```
init: starting sh
```

```
$
```

Q: What should we do to call our system call?

We are done! But...

```
$ make qemu
```

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
```

```
xv6 kernel is booting
```

```
hart 2 starting
```

```
hart 1 starting
```

```
init: starting sh
```

```
$
```

Q: What should we do to call our system call?

A: Let's make a user program!

Add a New Test Program

Extending usertests.c

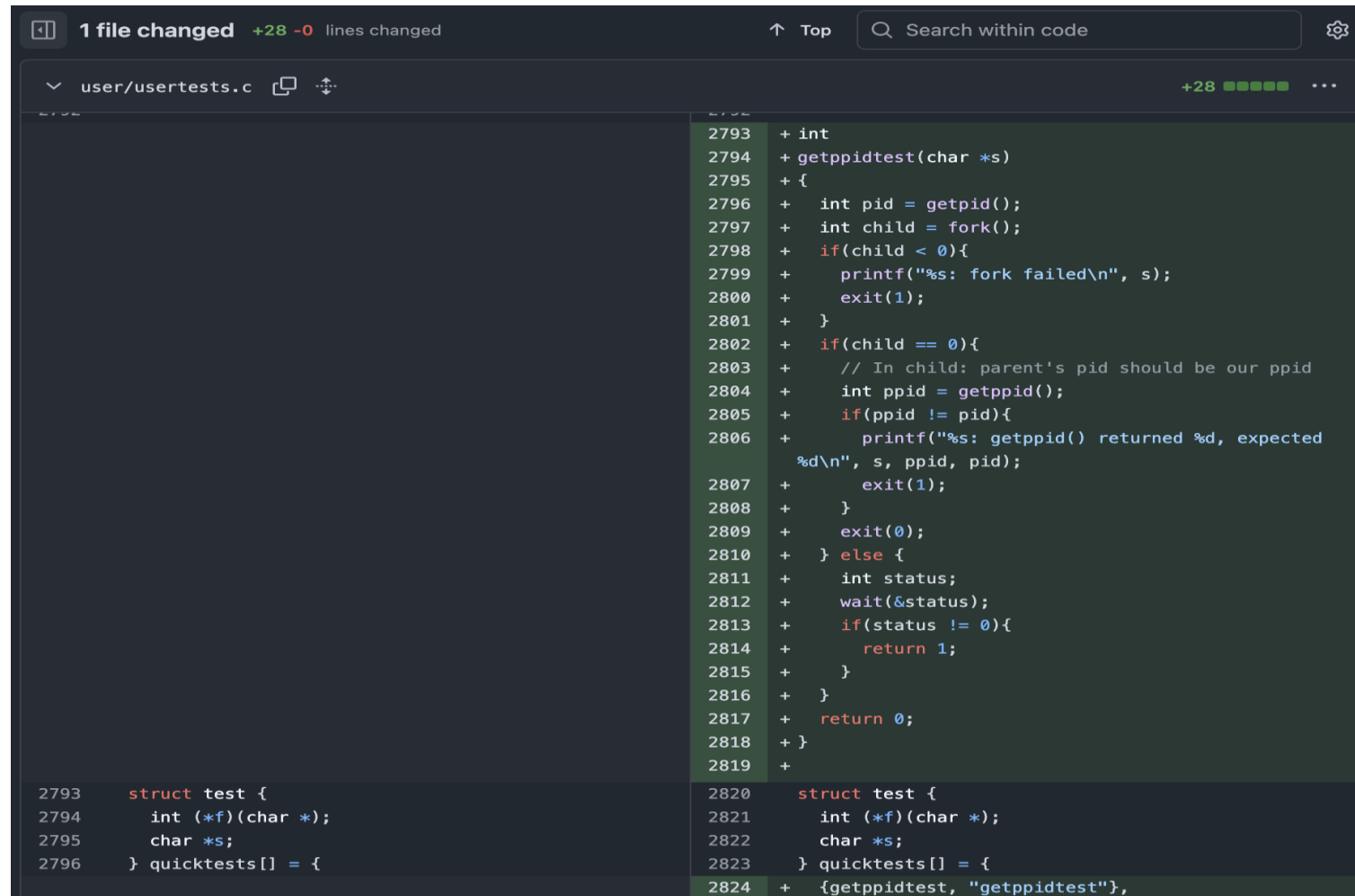
usertests

- In the previous lab, we added a user program (this approach also works here)
- This time, we will extend xv6's built-in usertests.
- usertests is a user program that tests whether xv6 is functioning correctly.
- Usage

```
$ usertests           # run all tests
$ usertests -q        # run quick tests only
$ usertests [testname] # run specific test
```

Step 8 – Extend usertests

- The test code is provided in the [sandbox/lab03](https://github.com/HanyangUniversity/Software-Platforms-Lab03)



The screenshot shows a code editor with a dark theme. At the top, a status bar indicates '1 file changed +28 -0 lines changed'. Below this, the file 'user/usertests.c' is open. The code is shown in two columns, with line numbers 2793 to 2824 visible. The code defines a new test function 'getppidtest' and adds it to the 'quicktests' array.

```
2793 + int
2794 + getppidtest(char *s)
2795 + {
2796 +     int pid = getpid();
2797 +     int child = fork();
2798 +     if(child < 0){
2799 +         printf("%s: fork failed\n", s);
2800 +         exit(1);
2801 +     }
2802 +     if(child == 0){
2803 +         // In child: parent's pid should be our ppid
2804 +         int ppid = getppid();
2805 +         if(ppid != pid){
2806 +             printf("%s: getppid() returned %d, expected
2807 +                 %d\n", s, ppid, pid);
2808 +             exit(1);
2809 +         }
2810 +     } else {
2811 +         int status;
2812 +         wait(&status);
2813 +         if(status != 0){
2814 +             return 1;
2815 +         }
2816 +     }
2817 +     return 0;
2818 + }
2819 +
2820 struct test {
2821     int (*f)(char *);
2822     char *s;
2823 } quicktests[] = {
2824 + {getppidtest, "getppidtest"},
```


Run usertests

```
$ make qemu
```

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
```

```
xv6 kernel is booting
```

```
hart 2 starting
```

```
hart 1 starting
```

```
init: starting sh
```

```
$ usertests getppidtest
```

```
usertests starting
```

```
test getppidtest: PASS
```

```
PASS ALL TESTS
```

```
$
```

Submission

- Changed Files

```
$ git status --short
```

```
M kernel/defs.h  
M kernel/syscall.c  
M kernel/syscall.h  
M kernel/sysproc.c  
M user/user.h  
M user/usertests.c  
M user/usys.pl
```

- Commit & Push

```
$ git add .                # Stage all changes  
$ git commit -m "lab03"    # Commit changes  
$ git push origin sandbox/lab03 # Push to remote repository
```

System Call Appendix in xv6

How to fetch arguments

Fetching Arguments

- Useful functions for fetching system call arguments are provided in **kernel/syscall.c**
- These functions validate system call arguments and ensure that pointers reference a valid address space for arrays or strings
 - For strings, also checks if the string is null-terminated
 - Returns 0 on success, -1 otherwise

```
int fetchaddr(uint64 addr, uint64 *ip);
int fetchstr(uint64 addr, char *buf, int max);
static uint64 argraw(int n);
void argint(int n, int *ip);
void argaddr(int n, uint64 *ip);
int argstr(int n, char *buf, int max);
```

Fetching Arguments

- Make sure that you use these functions; Do not access the pointer directly!
 - It is very important to make the code safe, and easy to debug
- If you want to fetch arguments in a specific form frequently, you can add another function to fetch that type of arguments
 - Ex) `argfd( )` function in `sysfile.c`

Thank you

